

★ WEEK 10 · PROBLEM SET

Finance Applications: Three Hands-On Problems

Score a Strategy · Purge a Timeline · Price the Luck

Each problem: Statement → Principles & Lab Guide → Hand Calculation → Claude Code Prompt

Small numbers you can compute with a pencil — then a metaprompt that lets Claude Code scale the same idea to real data. Hand first, machine second.

Every Problem Follows the Same Four Steps

Work top to bottom — do not skip the hand calculation, it is where the understanding lives

1 · Statement

the question

A concrete finance scenario with a tiny dataset you can see at a glance.

one slide

2 · Principles

the tools

The formulas involved and a short lab guide: what to compute, in what order.

one slide

3 · Hand Calc

pencil first

Work every number by hand in a table.
Small n , clean arithmetic, no calculator fear.

1–2 slides

4 · Claude Code

scale it up

A metaprompt: paste it into Claude Code and it builds the full lab on real data.

one slide

Hand first, machine second — you check the machine, never the other way around.

Where the Data Comes From

Public / in-script sources first — the prompts fetch free data; CSVs are hand-calc references or offline fallbacks

Problem	Primary source (public · no signup)	Provided fallback file
P1 · Sharpe	numbers on the slides — pure pencil work	fml_w10_p1_returns.csv (hand-calc)
P2 · Purged CV	§ 2 yfinance series (AAPL, 3y, free)	fml_w10_p2_timeline.csv (hand-calc)
P3 · Luck ladder	random signals generated in-script	fml_w10_prices_offline.csv (offline series)
Real-data extension	any yfinance ticker you like	—

Course rule: if data is free online (yfinance, FRED, OpenML) or generated in-script, the prompt uses it.

P1

PROBLEM 1 · SHARPE

Score the Strategy First

Five monthly returns — mean, wobble, annualize; then ask if the score can be trusted

A Strategy Reports Five Months

Compute its Sharpe ratio, monthly and annualized — then face the uncomfortable follow-up

Month	1	2	3	4	5
Return	+2%	−1%	+3%	0%	+1%

Helper

- Use the population std (divide by $n = 5$).
- $\sqrt{2} \approx 1.414 \cdot \sqrt{12} \approx 3.464$.
- Risk-free rate: assume 0 for this toy.

Your deliverables

- Mean return and the five deviations.
- Volatility σ , monthly Sharpe, annualized Sharpe.
- The follow-up: this was the best of 100 tries — does your answer change?

The Tool: Reward per Unit of Wobble

One fraction, then a square-root to change the clock

$$SR_{\text{monthly}} = \frac{\bar{r}}{\sigma_r} \quad SR_{\text{annual}} = \sqrt{12} \times SR_{\text{monthly}}$$

Reading it

- Numerator: the average monthly earn.
- Denominator: the typical wobble around that average.
- Annualizing multiplies by $\sqrt{12}$ because variance adds across months.

Lab guide

- Step 1: mean, then deviation of each month.
- Step 2: $\sigma = \sqrt{\text{mean of squared deviations}}$.
- Step 3: $SR = \text{mean}/\sigma$; then $\times \sqrt{12}$.

From Five Numbers to One Yardstick

Every step visible — mean, deviations, σ , and the two Sharpes

Step	Arithmetic	Result
Mean	$(2 - 1 + 3 + 0 + 1) / 5$	1%
Deviations	+1, -2, +2, -1, 0	squares: 1, 4, 4, 1, 0
Volatility σ	$\sqrt{(10/5)} = \sqrt{2}$	$\approx 1.41\%$
Sharpe (monthly)	$1 / 1.41$	0.71
Sharpe (annualized)	$0.71 \times \sqrt{12}$	≈ 2.45

$$\bar{r} = 1\%, \quad \sigma_r = \sqrt{2}\% \approx 1.41\% \Rightarrow SR_{\text{monthly}} = 0.71, \quad SR_{\text{annual}} \approx 2.45$$

Sharpe 2.45 — Impressive, Pending an Honest Audit

The number is correct; whether it MEANS anything is Problems 2 and 3

1%

Mean monthly earn —
the reward

1.41%

Monthly wobble σ —
the risk

2.45

Annualized Sharpe —
the advertisement

?

Whether it survives
leakage & luck checks
— the evidence

Why $\sqrt{12}$ and not 12

- Means grow with time; wobbles grow with $\sqrt{\text{time}}$.
- So the ratio scales by $\sqrt{12}$ monthly, $\sqrt{252}$ daily.

The follow-up answered

- Best of 100 tries? Luck's par is 1.75 (P3) — 2.45 still clears it.
- Best of 10,000? A very different conversation.

Scale It: Sharpe with Honest Error Bars

Paste the metaprompt — Claude Code scores a real series and shows how shaky Sharpe is

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) downloads AAPL with yfinance (period='3y',  
   auto_adjust=True); ret = pct_change().dropna();  
   (offline fallback: fml_w10_prices_offline.csv)  
2) verifies the toy first: returns [2,-1,3,0,1] in %,  
   population std -> confirm SR_monthly 0.71 and  
   annualized 2.45;  
3) computes AAPL's daily Sharpe and annualizes by  
   sqrt(252);  
4) bootstrap: resample the daily returns 2000 times,  
   recompute annualized Sharpe each time, and plot the  
   histogram with the 5th-95th percentile band;  
5) three-sentence verdict: how wide is the band, and  
   why a single Sharpe number deserves suspicion  
   before any deflation is even applied.
```

[OUTPUT] What to expect

```
toy check: 0.71 / 2.45  
           exactly  
AAPL 3y Sharpe: one number  
bootstrap band: wide -  
                often +/- 0.5 or more  
  
Lesson: even before luck  
and leakage, Sharpe wears  
big error bars.
```

P2

PROBLEM 2 · PURGED CV

Quarantine the Border Days

Ten days, three-day labels — decide each day's fate by hand

Ten Days, One Honest Split

Each day t has a label built from days t to $t+2$. Test window = days 5–7, embargo = 1 day. Classify every day

Day	1	2	3	4	5	6	7	8	9	10
Label uses days	1–3	2–4	3–5	4–6	5–7	6–8	7–9	8–10	9–11	10–12

The rules

- TEST: days 5, 6, 7.
- PURGE any training day whose label window touches the test set.
- EMBARGO 1 day immediately after the test window.

Your deliverables

- A verdict per day: train / purge / test / embargo.
- Why days 3–4 fall and day 8 sits out.
- The honest training count (and what naive CV would use).

The Tool: One Overlap Rule, One Buffer

If a label can see the test window, that row cannot teach the model

PURGE day t if its label window $[t, t+2]$ touches the test set; EMBARGO h days after it

Why purging matters

- Day 4's label is built from days 4–6 — days 5 and 6 are IN the test set.
- Training on it means grading the exam with the answers in hand.
- The leak is invisible in the code and glaring in live results.

Why embargo matters

- Day 8 starts right after the test — markets carry momentum and echoes.
- A short buffer stops test-period information seeping backward into training.
- One day here; a few days in practice.

Ten Verdicts, One Honest Split

Four days survive as training — naive CV would happily use eight

Day	1	2	3	4	5	6	7	8	9	10
Window	1–3	2–4	3–5	4–6	5–7	6–8	7–9	8–10	9–11	10–12
Touches test 5–7?	no	no	YES (5)	YES (5,6)	—	—	—	no	no	no
Verdict	train	train	PURGE	PURGE	test	test	test	EMBARGO	train	train

$$\text{test} = \{5, 6, 7\} \quad \text{purged} = \{3, 4\} \quad \text{embargo} = \{8\} \quad \text{honest train} = \{1, 2, 9, 10\}$$

Honest Training Set: {1, 2, 9, 10}

Half the 'usable' data was contaminated — and that is the normal case, not the exception

4 / 8

Honest training days
vs what naive CV
would use

2

Days purged — their
labels straddle the
border

1

Day embargoed — too
close behind the exam

↓ then ✓

Scores drop after
purging — and
become believable

Scaling up

- With 10-day labels and 5 folds, purging bites much harder.
- The longer the label horizon, the wider the quarantine zone.

The mindset

- Losing rows feels wasteful; keeping them is fraudulent.
- An honest 4-day score beats a leaky 8-day score, every time.

Scale It: Build Purged CV and Measure the Leak

Paste the metaprompt — Claude Code implements the quarantine and prices the difference

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) downloads AAPL with yfinance (period='3y'); builds  
   features = two lagged returns, label = next 3-day  
   cumulative return sign;  
   (offline fallback: fml_w10_prices_offline.csv)  
2) implements three scorers for LogisticRegression:  
   (a) shuffled KFold(5), (b) TimeSeriesSplit(5),  
   (c) purged time-ordered 5-fold: drop training rows  
   whose 3-day label window overlaps the test fold,  
   plus a 5-day embargo after it;  
3) prints the three accuracies in one table and the  
   number of rows purged per fold;  
4) reruns (c) with label horizons [1, 3, 10] and shows  
   how the purged-row count grows with the horizon;  
5) three-sentence verdict: rank the three scores by  
   trustworthiness and justify in this week's language.
```

[OUTPUT] What to expect

```
shuffled: highest score  
          (and least trustworthy)  
time-ordered: lower  
purged+embargo: lowest,  
                honest  
purged rows grow with the  
                label horizon  
  
Your 10-day quarantine,  
run on ~750 real days.
```

P3

PROBLEM 3 · LUCK'S PAR

How Good Is Pure Luck?

The best-of-N Sharpe that zero skill earns — computed, then used as a judge

Three Research Teams, One Question

Each team reports its best annualized Sharpe on 3 years of daily data ($T = 756$). How many tries did luck get?

Team	Strategies tested N	Best annualized Sharpe found	Claim
A	10	2.0	'a genuine edge'
B	100	2.0	'a genuine edge'
C	1,000	2.0	'a genuine edge'

Your deliverables

- Luck's par $\sqrt{(2 \ln N)/\sqrt{T}} \times \sqrt{252}$ for each N. Helpers: $\sqrt{(2 \ln 10)} \approx 2.15 \cdot \sqrt{(2 \ln 100)} \approx 3.04 \cdot \sqrt{(2 \ln 1000)} \approx 3.72 \cdot \sqrt{756} \approx 27.5$.
- A verdict per team on Sharpe 2.0, plus the honest reporting rule.

The Tool: the Expected Best of N Flukes

Extreme-value logic: the maximum of N noises grows like $\sqrt{2 \ln N}$

$$\mathbb{E} [\max SR_{\text{daily}}] \approx \frac{\sqrt{2 \ln N}}{\sqrt{T}} \quad SR_{\text{annual}} = \sqrt{252} \times SR_{\text{daily}}$$

Reading it

- Each random strategy's Sharpe is noise of size $\sim 1/\sqrt{T}$.
- The best of N such noises sits $\sim \sqrt{2 \ln N}$ deviations out.
- Slow logarithm — but it never stops climbing.

Lab guide — per team

- Step 1: $\sqrt{2 \ln N}$ from the helpers.
- Step 2: divide by $\sqrt{756} \rightarrow$ daily par.
- Step 3: $\times \sqrt{252} \rightarrow$ annualized par; compare to 2.0.

The Ladder, Then the Judgments

Same Sharpe 2.0 — three completely different verdicts

Team	$\sqrt{2 \ln N}$	Daily par ($\div 27.5$)	Annualized par ($\times \sqrt{252}$)	Sharpe 2.0 verdict
A · N=10	2.15	0.078	≈ 1.24	well ABOVE par — promising
B · N=100	3.04	0.110	≈ 1.75	above par — marginal, dig deeper
C · N=1,000	3.72	0.135	≈ 2.15	BELOW par — luck explains it

$N=10: 1.24$ $N=100: 1.75$ $N=1000: 2.15$ (luck's par score, $T = 756$)

The Trial Count IS Part of the Result

A Sharpe without its N is an advertisement, not evidence

1.24

Luck's par after 10
honest tries — 2.0
clears it

2.15

Luck's par after 1,000
tries — 2.0 drowns in it

$\ln N$

Why quitting at 10
tries is a virtue: the
par never stops
climbing

$SR + N$

The honest report:
Sharpe, trial count,
and the DSR together

The DSR connection

- The DSR formalizes this exact comparison into a probability of skill.
- Par-beating margin $\div$ estimation noise $\rightarrow$ a number between 0 and 1.

The cultural fix

- Log every trial before running it — N is a fact, not a memory.
- Week 4's factor zoo was this problem at industry scale.

Scale It: Simulate the Ladder and Judge a Winner

Paste the metaprompt — Claude Code races luck against the formula and issues verdicts

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:
1) downloads AAPL with yfinance (period='3y');
   ret = pct_change().dropna();
   (offline fallback: fml_w10_prices_offline.csv)
2) for N in [10, 100, 1000]: generate N random +/-1
   signals, compute each annualized Sharpe on ret,
   record the best; repeat 200 times and average;
3) plot simulated best-of-N vs the formula
    $\sqrt{2 \ln(N)} / \sqrt{T} \cdot \sqrt{252}$  on the same axes;
4) verdict engine: a function that takes (sharpe, N)
   and prints ABOVE/BELOW luck's par; run it on
   (2.0, 10), (2.0, 100), (2.0, 1000);
5) three-sentence verdict: how close simulation and
   formula track, and what to demand from anyone
   reporting a backtest Sharpe.
```

[OUTPUT] What to expect

```
simulation hugs the
  sqrt(2 ln N) curve
(2.0, 10): ABOVE par
(2.0, 100): above, thin
(2.0, 1000): BELOW par
```

Your three-row ladder,
reproduced by 200
Monte-Carlo rounds.

Problem Set — Three Things Your Hands Now Know

You computed every number yourself; the code only repeats what you already understand

1

Sharpe by hand: 0.71 monthly, 2.45 annualized

Mean over wobble, then $\sqrt{12}$ — and even a correct 2.45 is an advertisement until audited.

2

Purged CV: honest train = {1, 2, 9, 10}

Days 3–4 purged for label overlap, day 8 embargoed — half the data was contaminated, and that's normal.

3

Luck's par ladder: 1.24 → 1.75 → 2.15

The same Sharpe 2.0 is promising after 10 tries and pure luck after 1,000 — the trial count IS the result.

Hand calculation first — then let Claude Code scale the same idea to real data.